

AI – TECH

SKRYPT DO LABORATORIUM

UCZENIE MASZYNOWE

LABORATORIUM 4:

Metoda gradientu prostego, perceptron

dr Tomasz Neumann



1. Opis ćwiczenia

Wymagania wstępne:

Osiągnięcie celów ćwiczenia będzie możliwe poprzez właściwe przygotowanie się studenta do zajęć.

Będzie to możliwe poprzez:

- powtórzenie wiedzy nabytej w czasie wykładów,
- zapoznanie się (przed ćwiczeniem) z instrukcją do ćwiczenia ilustrującą podstawowe zagadnienia z zakresu tematyki ćwiczenia.

Ponadto wymagana jest od studenta:

- podstawowa umiejętność programowania,
- podstawowa znajomość języka Python,
- podstawowa znajomość analizy danych,
- umiejętność twórczego myślenia.

Wyżej wymienione wymagania bazują na wiedzy i umiejętności jakie studenci powinni zdobyć w czasie edukacji na I poziomie studiów jak i w ramach przedmiotu „Języki programowania dla sztucznej inteligencji”. W przeciwnym przypadku studenci powinni nadrobić samodzielnie stosowny materiał.

Stanowisko laboratoryjne powinno być wyposażone w komputer z dostępem do sieci komputerowej oraz z następującymi zasobami:

- przykładowe programy (kody programów) podane w załączniku,
- zainstalowany interpreter języka Python w wersji co najmniej 3.6 z możliwością doinstalowania bibliotek standardowych za pomocą pip lub anaconda (najlepiej w ramach dedykowanego środowiska wirtualnego),
- zainstalowany Jupyter Notebook,
- dostęp do rozwiązywania chmurowego Google Colab,
- inne: przeglądarka WWW.

Na komputerze dostępne powinny być zainstalowana instrukcja do ćwiczenia w wersji elektronicznej oraz pliki testowe. Przed zajęciami stacjonarnymi, prowadzący powinien sprawdzić czy dostępne są na stanowisku wymagane zasoby i programy.

Cele ćwiczenia:

Celem ćwiczenia jest praktyczna demonstracja wiedzy teoretycznej, przedstawionej w ramach wykładów. Realizując ćwiczenie laboratoryjne studenci zdobędą szereg nowych umiejętności w zakresie stosowania metody gradientu prostego oraz wykorzystania klas SGDClassifier i SGDRegressor jak i wykorzystania wysokiego poziomu API Keras oraz interpretacji uzyskanych wyników w typowych problemach uczenia maszynowego.

Spodziewane efekty kształcenia - umiejętności i kompetencje:

Student po zajęciach laboratoryjnych będzie posiadał zwiększoną umiejętność przygotowania danych oraz zastosowanie metody gradientu prostego z pakietu scikit-learn oraz Keras API oraz będzie w stanie zinterpretować uzyskane wyniki w standardowych problemach klasyfikacji i regresji.

Metody dydaktyczne:

Student w pierwszej kolejności realizuje zadania przykładowe podczas których przygotowuje odpowiednie środowisko programistyczne, poprzez które testuje poszczególne elementy pracy z danymi do których można wykorzystać metodę gradientu prostego na podstawie przykładowej implementacji jak i w pakietach scikit-learn oraz Keras API. Następnie, na podstawie posiadanej wiedzy i zdobytej umiejętności z zadań demonstracyjnych, realizuje przydzielone zadania. Wszystkie kody źródłowe programów wpisuje w dokumencie elektronicznym (lub pokazuje prowadzącemu), który jednocześnie staje się sprawozdaniem z ćwiczenia laboratoryjnego.

Materiały wprowadzające i pomocnicze:

- Python – dokumentacja języka <https://docs.python.org/pl/3/>
- SciKit-Learn – dokumentacja do podstawowej biblioteki uczenia maszynowego stosowanej w języku Python: <https://scikit-learn.org/stable/tutorial/basic/tutorial.html>
- Keras – API przeznaczone dla ludzi i maszyn, oferujący spójne i proste interfejsy, minimalizując liczbę działań użytkownika wymaganych w typowych przypadkach, prezentując jednocześnie

jasne i przydatne komunikaty o błędach. Zawiera także obszerną dokumentację, przewodniki i przykłady dla programistów - <https://keras.io/>

- Jupiter notebook – narzędzie ułatwiające implementację kodu, uruchamianie oraz testowanie: <https://realpython.com/jupyter-notebook-introduction/>
- Google Colab – narzędzie umożliwiające pracę z kodem w notebooku Jupiter oraz jego uruchamianie w chmurze (konieczność posiadania konta Google): <https://colab.research.google.com/github/cs231n/cs231n.github.io/blob/master/python-colab.ipynb>

Zasady oceniania/warunek zaliczenia ćwiczenia

Ocenie podlegać będzie realizacja poszczególnych zadań (fragmenty kodu, analizy i wnioski) w ramach danego ćwiczenia laboratoryjnego. Dodatkowo w czasie realizacji ćwiczenia przydzielone zostaną studentowi dodatkowe zadania. Poszczególnym zadaniom przydzielono liczbę punktów jakie można zdobyć za ich wykonanie. Maksymalna liczba punktów wynosi 10.

Wykaz literatury podstawowej do ćwiczenia:

1.	Materiały z wykładu do przedmiotu „Języki programowania dla sztucznej inteligencji”
2.	Materiały z wykładu do przedmiotu „Uczenie maszynowe”
3.	Marcin Szeliga, Praktyczne uczenie maszynowe, PWN 2019
4.	M. Gagolewski, M. Bartoszek, A. Cena, Przetwarzanie i analiza danych w języku Python, PWN, 2016
5.	Sebastian Raschka, Python. Uczenie maszynowe, Helion 2017
6.	Aurelien Geron, Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow, 2019

2. Przebieg ćwiczenia

L.p.	Zadanie
1.	Zapoznanie się z instrukcją laboratoryjną (przed ćwiczeniem)
2.	Wprowadzenie kierownika ćwiczenia. Zapoznanie się uczestników ze stosowanymi programami, przegląd dokumentacji (5 min.)
3.	Stochastyczny gradient prosty, batch i mini-batch (25 min.)
4.	Perceptron, SGDClassifier (45 min.)
5.	SGDRegressor (35 min.)
6.	Wykorzystanie API wysokiego poziomu - Keras (20 min.)
7.	Przesłanie/pokazanie zbioru dokumentów/programów do oceny (5 min.)

Uwagi

PRZED przystąpieniem do ćwiczenia należy utworzyć własny, unikalny katalog w lokalizacji wskazanej przez prowadzącego, a następnie należy tam umieścić pliki testowe oraz własne implementacje kodu programów dokonujących rozwiązania zadania. Należy pracować wyłącznie na wykonanej kopii plików.

3. Wprowadzenie do ćwiczenia

Metoda gradientu prostego (gradient descent) polega na wielokrotnym poprawieniu wartości parametrów w celu zminimalizowania funkcji kosztu. W metodzie tej na początku inicjujemy losowymi wartościami wektor parametrów θ , a następnie za pomocą 'małych kroków' dążymy do zredukowania funkcji kosztu (np. MSE), aż do momentu uzyskania zbieżności algorytmu z wartością minimalną tejże funkcji. W metodzie tej ważny jest rozmiar poszczególnych 'kroków', który określany jest za pomocą hiperparametru określanego jako współczynnik uczenia (learning rate). Przy wyborze tej wartości należy mieć na uwadze, aby nie była zbyt mała, gdyż algorytm wówczas będzie potrzebował bardzo dużo

przebiegów do osiągnięcia oczekiwanej wartości, a co za tym idzie będzie to czasochłonne. Zbyt duża wartość może doprowadzić natomiast do sytuacji w której nastąpi przeoczenie minimum, jednocześnie znajdując się po drugiej stronie tej doliny, nawet z wartością większą niż początkową. W konsekwencji może doprowadzić do rozbieżności pracy algorytmu, a co za tym idzie, nie będzie można znaleźć rozwiązania problemu. Należy także mieć na uwadze wybór funkcji kosztu do minimalizacji, gdyż nie wszystkie mają 'ładny kształt', posiadając minima lokalne, minima globalne i wypłaszczenia, co w przypadku różnych losowych inicjacji początkowych, może nie doprowadzić do osiągnięcia globalnego minimum. W przypadku modelu regresji liniowej funkcja kosztu jest wypukła, stąd nie ma minimów lokalnych (wybór dwóch różnych punktów na jej przebiegu, nigdy nie przetnie się z tą krzywą), stąd istnieje jedno minimum globalne. Dlatego metoda gradientu prostego pozwala na zbliżenie się do minimum globalnego przy odpowiedniej wartości współczynnika uczenia oraz po wystarczająco długim czasie. W uczeniu maszynowym metoda spadku gradientu jest techniką optymalizacji używaną do obliczania parametrów modelu (współczynników i ich odchyleń standardowych) dla algorytmów takich jak regresja liniowa, regresja logistyczna, sieci neuronowe itp. W tej technice wykonuje się wielokrotnie iteracje przez zestaw uczący, aktualizując parametry modelu zgodnie z gradientem błędu w odniesieniu do zbioru uczącego. W zależności od wielkości zbioru danych treningowych branych pod uwagę podczas aktualizacji parametrów modelu, możemy wyróżnić trzy podejścia:

- Batch Gradient Descent - parametry są aktualizowane w oparciu o cały zestaw danych uczących (wykonanie pojedynczej aktualizacji zajmuje dużo czasu);
- Stochastic Gradient Descent - parametry są aktualizowane po obliczeniu gradientu błędu w oparciu o pojedynczy przykład treningowy (zapętlenie zestawu uczącego, a tym samym nie można wykorzystać w pełni prędkości obliczeń związanej z wektoryzacją kodu);
- Mini-Batch Gradient Descent - parametry są aktualizowane po obliczeniu gradientu błędu w odniesieniu do podzbioru zbioru uczącego (może dokonywać szybkich aktualizacji wykorzystując także potencjał wektoryzacji kodu, jednak należy uważać na szybkie i chwilowe aktualizacje parametrów modelu w zależności od przyjętego rozmiaru paczki danych).

Tak więc, Mini-Batch Gradient Descent stanowi kompromis między szybką zbieżnością a zakłóceniami związanymi z aktualizacją gradientu, co czyni go bardziej elastycznym i niezawodnym algorytmem. Część praktyczna będzie wykonana przy pomocy skryptu *lab4_ex01.ipynb*.

Perceptron stanowi jedną z najprostszych architektur SSN, zaproponowaną w 1957 roku przez Franka Rosenblatta. Jego podstawą jest nieco zmodyfikowany sztuczny neuron, zwany progową jednostką logiczną (ang. Threshold Logic Unit — TLU) lub liniową jednostką progową (ang. Linear Threshold Unit — LTU). Wartościami wejść/wyjść są liczby (a nie stany binarne), a każde połączenie ma przyporządkowaną wagę. Jednostka TLU wylicza ważoną sumę sygnałów wejściowych, a następnie zostaje użyta funkcja skokowa (najczęściej można spotkać rozwiązanie z funkcją skokową Heaviside'a lub funkcję *signum*) wobec tej sumy, dająca ostateczny wynik. Pojedyncza jednostka TLU może być używana w prostych zadaniach klasyfikacji. Oblicza liniową kombinację danych wejściowych i jeżeli wynik przekracza określony próg, to klasyfikuje przykład do klasy pozytywnej, jeśli zaś rezultat nie przekroczy progu — do klasy negatywnej. Przykładową implementację pojedynczej sieci TLU zawiera moduł *scikit-learn* w klasie *Perceptron*.

Okazuje się, że klasa *Perceptron* jest równoważna stosowaniu klasy *SGDClassifier* z następującymi parametrami: *loss="perceptron"*, *learning_rate="constant"*, *eta0=1* (współczynnik uczenia) oraz *penalty=None* (brak regularyzacji). Na tym etapie należy zwrócić uwagę, że w przeciwieństwie do klasyfikatorów regresji logistycznej, perceptrony nie wyliczają prawdopodobieństwa przynależności do klasy: określają one prognozy na podstawie wyznaczonego progu. Jest to jeden z powodów, dla których warto wybierać klasyfikatory regresji logistycznej przed perceptronami.

Przy pomocy notebooka *lab4_ex02.ipynb* możliwe będzie wytrenowanie klasyfikatora stochastycznego spadku wzdłuż gradientu (Stochastic Gradient Descent — SGD z klasy *SGDClassifier*). Algorytm ten cechuje się możliwością wydajnego przetwarzania bardzo dużych zestawów danych. Wynika to częściowo z faktu, że klasyfikator SGD przetwarza poszczególne przykłady uczące niezależnie od siebie, po jednym naraz - z tego powodu nadaje się on również do uczenia przyrostowego. Istotną rzeczą w tym ćwiczeniu będzie wytrenowanie klasyfikatora za pomocą krosvalidacji krzyżowej oraz ocena klasyfikatora na podstawie macierzy pomyłek, zależności precyzji od czułości oraz krzywej ROC.

Kolejny skrypt pozwoli poznać dalsze zakątki pakietu scikit-learn API, które udostępnia klasę SGDRegressor do implementacji metody SGD dla problemów regresji. Regresor SGD stosuje regularyzowany model liniowy uczeniem SGD w celu zbudowania estymatora. Są to regulatory L1, L2 lub Elastic Net dodane do funkcji strat w celu zmniejszenia parametrów modelu. Regresor SGD działa dobrze z wielkoskalowymi zbiorami danych. Spróbujmy więc dopasowywać i przewidywać dane regresji przy użyciu klasy SGDRegressor modułu Scikit-learn przy użyciu pliku *lab4_ex03.ipynb*.

Jako ostatek zostanie wykorzystane wysokiego poziomu KERAS API umożliwiające zaimplementowanie problemu w prosty i przejrzysty sposób - szczegóły znajdują się w pliku *lab4_ex04.ipynb*.

4. Forma i zawartość sprawozdania

Ocenie podlegać będzie realizacja poszczególnych zadań (fragmenty kodu, analizy i wnioski) w ramach danego ćwiczenia laboratoryjnego. Raport powinien być złożony w formacie PDF i przedstawiać w sposób jasny i przejrzysty wykonane eksperymenty i analizy oraz opatrzone powinny być odpowiednimi wnioskami końcowymi.

Dodatki:

Pliki instruktażowe oraz pliki z danymi niezbędne do wykonania ćwiczenia laboratoryjnego:

- 1) *lab4_ex01.ipynb*
- 2) *lab4_ex02.ipynb*
- 3) *lab4_ex03.ipynb*
- 4) *lab4_ex04.ipynb*